Exploring ML Models for Regression and Classification Report

Author: Arslonbek Ishanov

1. Introduction

With the rapid advancements in the Artificial Intelligence (AI) industry, its adoption has become more widespread. Consequently, technological giants such as Amazon, Google, Microsoft, OpenAI and Apple announced investments of $40 billion, $75 billion, $80 billion, $100 billion and $500 billion (USD), respectively. Moreover, AI finds newer applications in everyday life, ranging from ChatGPT to online shopping.

The industry could also benefit other sectors. For example, Machine Learning (ML) algorithms could evaluate the price of a house based on its location, and a household's income or identify if a person would have been likely to survive the Titanic crash. This study applied Lasso Regressor (LR), Random Forest (RF), XGBoost (XGB) to the former case, and K-Nearest Neighbours, Support Vector Machine (SVM), Neural Network – to the latter. While all models fit the datasets well, RF performed best in the regression task, while Neural Network did best in the classification task.

2. Regression

Accurately estimating the price of a house can be challenging due to the number of factors that could affect it. The modified version of the California Housing dataset tracks nine metrics that tend to affect the price of a house, alongside the price of a house it was sold for. However, before ML can be applied to the dataset, it needs to be cleaned and preprocessed.

2.1. Preprocessing

Data cleaning and preprocessing are essential as these steps can fill in the missing values, normalise the data for all the features to carry the same weight, thus improving the performance of a model. The dataset had the dimensions of 10 by 1,000 (or 9 by 1,000 without the target variable), consisting of the following columns:

- Longitude
- Latitude
- Housing_median_age
- Total_rooms
- Total_bedrooms
- Population
- Households
- Median_income
- Median_house_value
- Ocean_proximity

To begin with, the data was loaded into the environment and split into Train and Test datasets in the ratio 810:190. Next, missing values were filled for each column with the median values for numerical columns from the Train dataset, and the most frequent objects in the object groups. The same values were applied to respective columns with null values in Test dataset. The order of these operations helps to avoid data leakage since calculating median / most frequent values in the

original dataset would involve the testing data, allowing the training to indirectly involve the test data. Finally, duplicate rows in each dataset were removed.

Lastly, the target variable was separated from the features and stored in the Y-variables (Y_train, Y_test), while the features were assigned to X-variables (X_train, X_test). This would make clear to the AI what it is trying to predict.

Additionally, some extra features were engineered. Essentially, some original features were multiplied/divided by each other to obtain new values shown below:

- Room_density $= \frac{\text{Total\_rooms}}{\text{Households}}$ (how spacious each room is).
- Bedroom_density $= \frac{\text{Total\_bedrooms}}{\text{Households}}$ (how spacious each bedroom is).
- Pop_density $= \frac{\text{Population}}{\text{Households}}$ (how many people live in a house on average).
- Income_bedroom $=$ Income $\times$ Total_bedrooms (typically, wealthier households have more bedrooms).

Feature engineering helps capture the domain knowledge by painting a fuller picture, and consequently, improve a model's performance. Feature engineering was applied to X_train and X_test separately, and the new results were stored in X_train_extra and X_train_extra.

In contrast, dimensionality reduction focuses on reducing the size of the data. It focuses on preserving as much information as possible while reducing the number of unnecessary features. This lowers overfitting (when the model memorises data instead of learning how to handle it), improves computational efficiency, and removes multicollinearity (when features are highly inter-correlated). In fact, Figure 1 shows that Total_rooms, Total_bedrooms, Population and Households are all highly inter-correlated, with the correlation index above 0.85. By randomly removing three of these features, the dimensionality of the new dataset (stored in X_train_small and X_test_small) is reduced by a third, and the new correlation matrix is displayed in Figure 2.

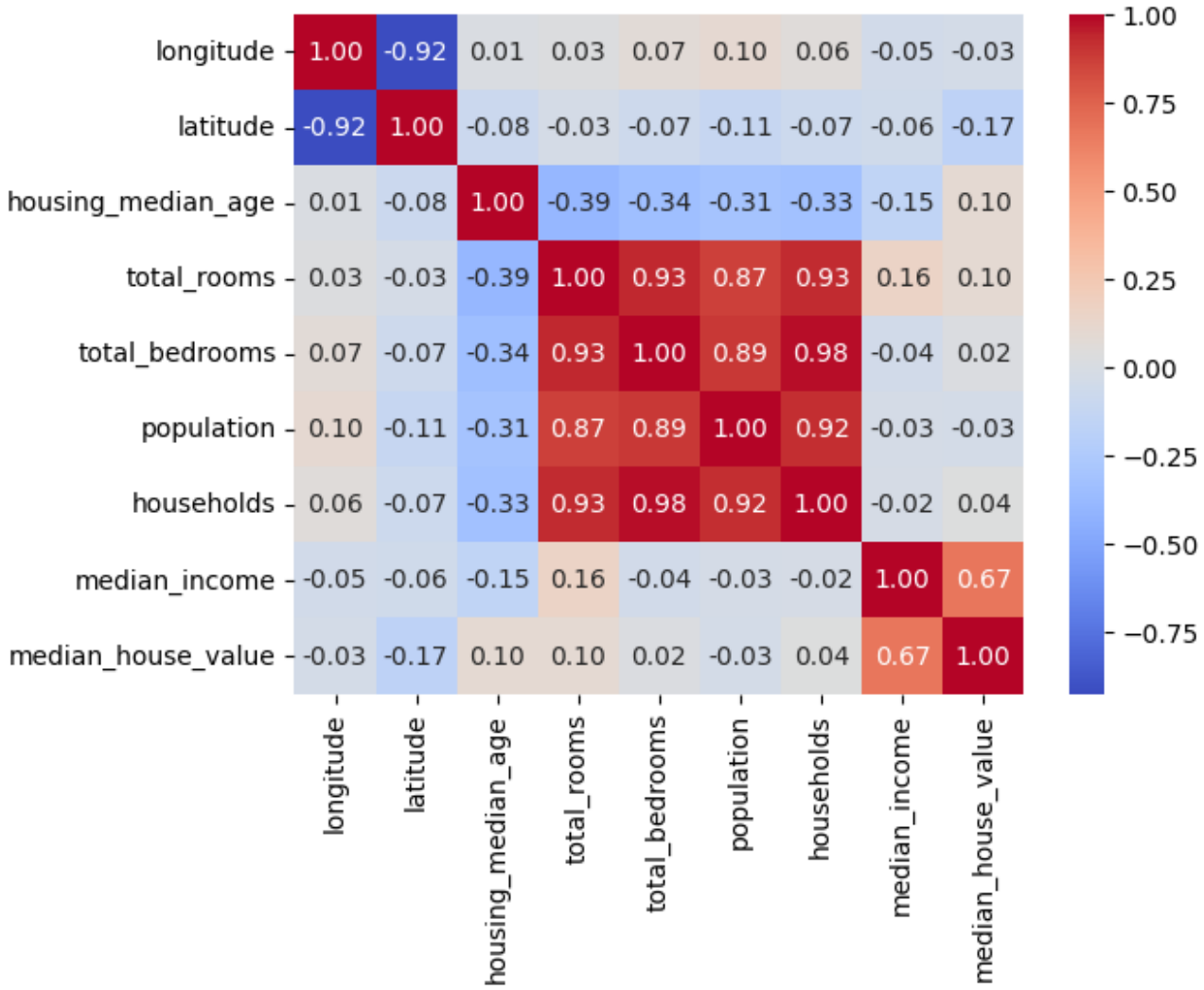

Figure 1. The correlation matrix of the original dataset.

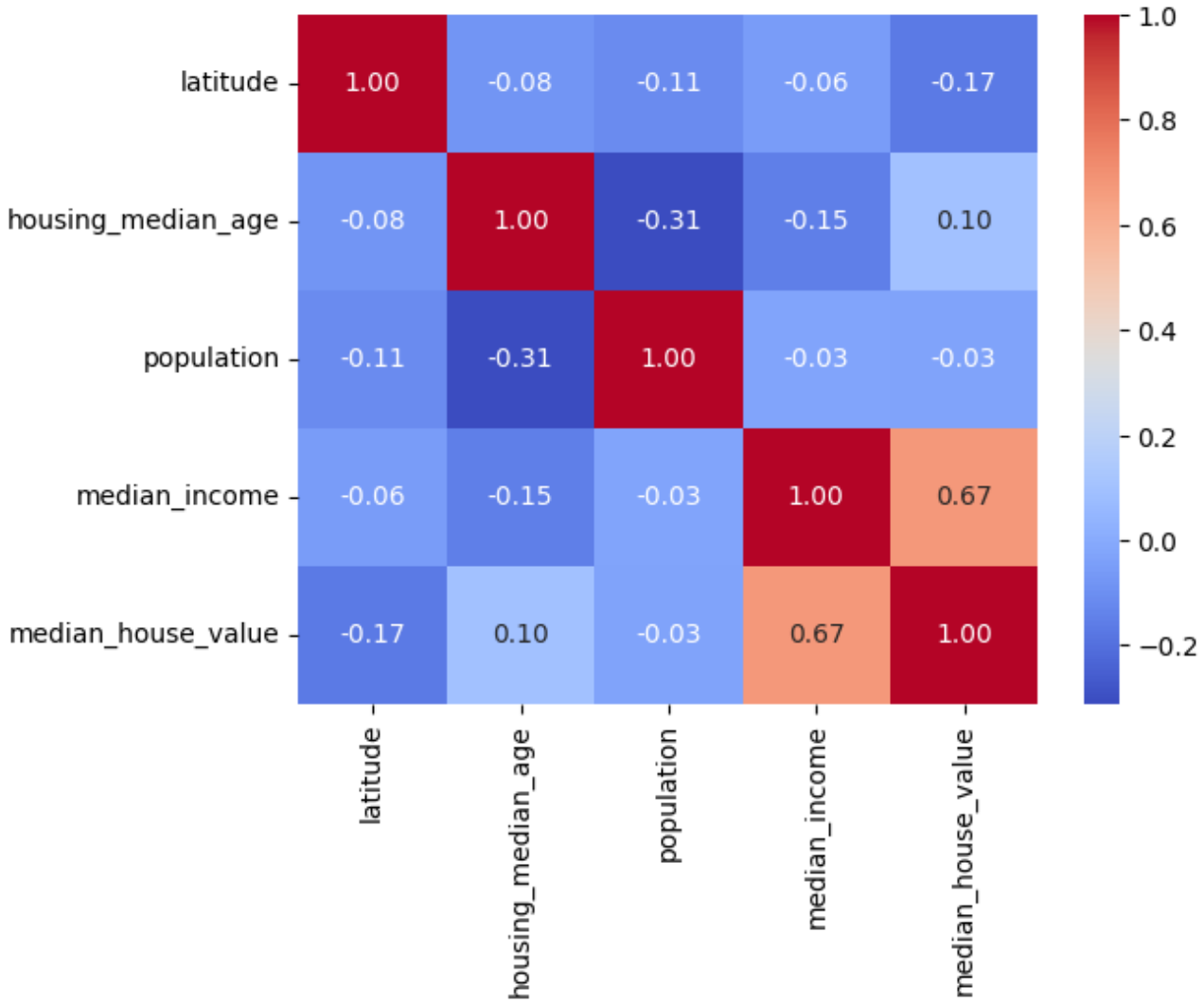

Figure 2. New correlation matrix of the ”small” dataset.

Finally, it is worth noting that data pre-processing is also supposed to address the outliers. As can be seen in Figure 3, several features have multiple outliers. However, these are actual correct data points that exist in real life. Thus, the outliers were not addressed in this case study.

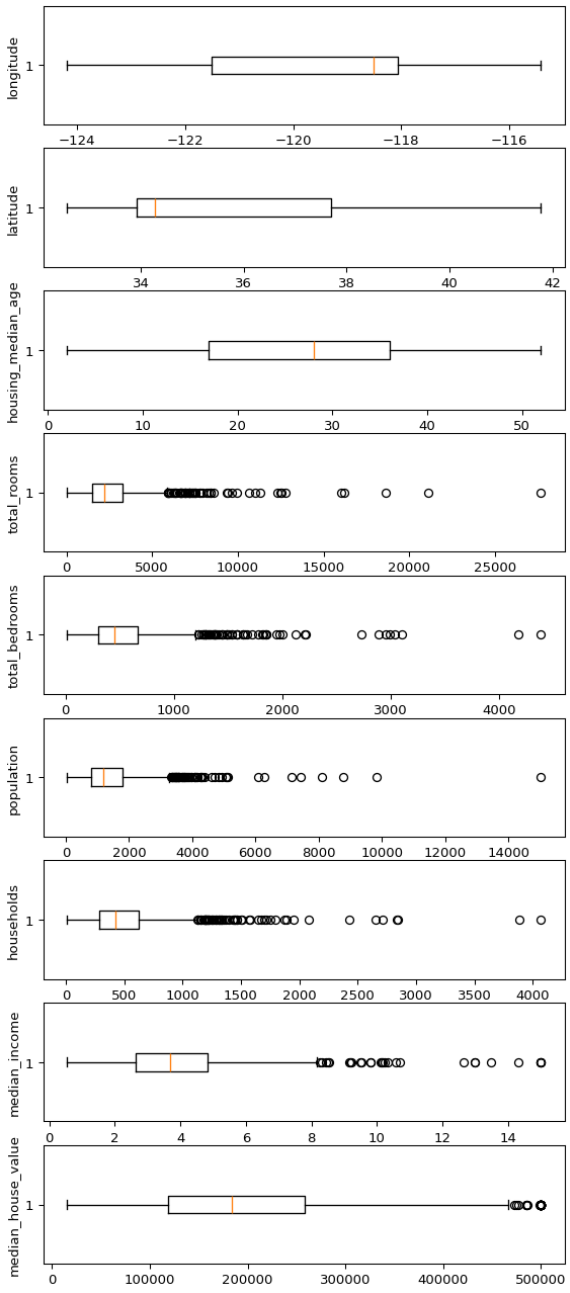

Figure 3. The outliers in each numerical column in the original dataset.

2.2. Methodology

While LR and RF are different models, they are great candidates for solving a regression task. They will be used as baseline models and will be compared to XGB, which will be the main model.

The LR, also known as L1 Regularisation, is similar to a regular Linear Regressor, which begins by trying to find a straight line/hyperplane that best fits the given data. However, instead of minimising the error, LR penalises the model for having large weights, which is based on the sum of absolute values of the coefficients. Therefore, it shrinks less important features to zero, basically, removing them. The formula for LR to minimise is:

$$\text{Loss} = \sum (y - \hat{y})^2 + \lambda \sum |w|$$

Where:

- y = actual value
- $\hat{y}$ = predicted value
- w = feature weights
- λ = penalty strength

Random forest builds multiple decision trees, each one learns from the data and makes a prediction for each data point in the test dataset, and RF averages the results, thus producing the outcome. This approach reduces overfitting and produces a stabler model compared to a Decision Tree algorithm. RF's mathematical formula to average the predictions is:

$$\hat{y} = \frac{1}{n} \sum_{i=1}^{n} f_i(x)$$

Where:

- $f_i(x)$ = prediction of the i-th decision tree
- n = number of decision trees in a Random Forest model

Finally, XGB is a sequential ML model which, similarly to RF, builds decision trees. However, the main difference is that each decision tree is built sequentially, with each tree correcting the error of the previous ones by trying to minimise the objective function:

$$\mathcal{L} = \sum_{i=1}^{n} \text{Loss}_i + \sum_{t=1}^{T} \text{Reg}_t$$

Where:

- Loss_i = the loss for each data point
- Reg_t = the regularisation term for tree t

The prediction for each iteration/tree is calculated as

$$\hat{y}_t(x) = \hat{y}_{t-1}(x) + \eta \cdot \text{Tree}_t(x)$$

Where:

- $\hat{y}_{t-1}(x)$ = the previous prediction (i.e., tree)
- η = the learning rate
- $\text{Tree}_t(x)$ = the prediction from the t-th tree

Due to the intricacy of XGB and the non-linear relationship of the data, XGB was chosen as the main model. Additionally, it applies L1 and L2 regularisation, as well as decision trees, meaning that it is an ensemble method, partially combining RF and LR. Finally, the base models do not conduct error-correction as rigorously as XGB.

2.3. Experiments

The experiments involved two main stages. The first one would introduce simple, basic models of each type with the default parameters. This would signify how well each model can handle the data. The second step would involve fine-tuning the algorithms using GridSearchCV or Optuna. The former was chosen due to LR and RF's incompatibility with being run on a Graphical Processing Unit (GPU), and the latter – because it can run on GPU which significantly improves the optimisation time.

2.3.1. Experimental Settings

The basic models we created with the default parameters which can be found in their documentation.

GridSearchCV automated the process of adjusting alpha (regularisation) in LR to minimise the objective function and improve the performance of LR.

Similarly, GridSearchCV fine-tuned RF, albeit with significantly more parameters, as shown below:

- n_estimators: 50, 100, 150, 200, 250, 300 – sets the number of trees.
- max_depth: None, 10, 20, 30 – sets the maximum depth of each tree.
- min_samples_split: 2, 5, 10 – minimum number of samples required to split an internal node.
- min_samples_leaf: 1, 2, 4 – minimum number of samples required to be considered a leaf node.
- max_features: 'sqrt', 'log2', None – number of features to consider when deciding the best split.
- bootstrap: True, False – whether the model samples with replacement.

Identifying the best combination of these factors manually, would take a long time. However, GridSearchCV significantly accelerates this process. Its major drawback is its incompatibility with the GPU, which could significantly accelerate the process even further.

However, Optuna overcomes such an issue. Moreover, it uses the Bayesian optimisation method, a probabilistic model which predicts the best set of hyperparameters, iteratively improving the results. Optuna optimised the following parameters in XGB:

- max_depth: 3, 15 – the maximum depth of a tree.
- learning_rate: 0.01, 0.3 – the contribution of each tree to the final prediction.
- n_estimators: 50, 500 – the total number of trees/iterations.
- subsample: 0.5, 1.0 – the fraction of samples to be used for each tree.
- colsample_bytree: 0.5, 1.0 – fraction of features to be used for each tree.
- min_child_weight: 1, 10 – the minimum sum of instance weights required in a child.

In addition, all optimisations used five-fold cross-validation to avoid overfitting, enhance model evaluation reliability, and improve the use of limited data and model stability. Finally, random_state was set to 42 to ensure the reproducibility of the results.

2.3.2. Results

Model \ Metric	MAE	MSE	RMSE	R^2
Lasso Regressor	53764.9531	5022930825.4830	70872.6381	0.6958
Random Forest	52980.4200	5076905004.3424	71252.4035	0.6925
XGBoost	**52240.2344**	**4932249600.0000**	**70229.9765**	**0.7012**

Table 1. Performance of the regression models on the original dataset.

Model \ Metric	MAE	MSE	RMSE	R^2
Lasso Regressor	53761.4055	5021922729.6427	70865.5257	0.6958
Random Forest	57803.9929	6019924841.3540	77588.1746	0.6354
XGBoost	**48990.7109**	**4534239744.0000**	**67336.7637**	**0.7254**

Table 2. Performance of the fine-tuned regression models on the original dataset.

Fitting the models onto _small and _extra datasets did not produce any noticeable changes. Therefore, the results were omitted.

Four evaluation metrics were chosen to assess the performance of each model comprehensively.

- **MAE** – Mean Absolute Error – measures the absolute differences between predicted and actual values, providing a clear, interpretable error.
- **MSE** – Mean Squared Error – similar to MAE but squared, emphasising larger errors.
- **RMSE** – Root Mean Squared Error – the square root of MSE, providing the error in the same unit as the target variable.
- **R^2** – Coefficient of Determination – represents the proportion of variance in the target variable explained by the features.

2.3.3. DISCUSSION

As Tables 1 and 2 showcase, the main model performed better before and after fine-tuning, albeit with a small lead in the first case. As the theory suggested, XGB's use of L1 and L2 regularisation and sequential building of decision trees to correct its past mistakes, made it outperform the base models, with a higher advantage after fine-tuning. While it is far from perfect, the model generalises well and has a rather high Coefficient of Determination (i.e., better explains the variance of the house prices). Considering that the price of a house is measured in hundreds of thousands of dollars, an error of about 49,000 dollars is rather small, and is far superior to that of RF and LR.

Notably, it is impossible to predict a house price with 100% accuracy due to human nature; XGB can provide an approximate evaluation of a house in California. In the future, the work can be expanded to other cities, the dataset could be extended to include more homes and more features, and the model could be combined with other models to improve its performance.

3. Classification

Classification refers to a supervised ML method where a model aims to predict the correct label of a provided input data point. This can be demonstrated on the modified version of the given "Titanic" dataset. Naturally, the data has to be pre-processed.

3.1. Pre-processing

The given dataset consists of 9 features:

- **Pclass** – Ticket class: 1 – First class, 2 – Second, 3 – Third.
- **Name** – Passenger's name.
- **Sex** – Passenger's gender.
- **Age** – Age in years.
- **SibSp** – Number of siblings/spouses aboard the Titanic.
- **Parch** – Number of parents/children aboard the Titanic.
- **Ticket No.** – Ticket number.
- **Fare** – Passenger's fare.
- **Embarked** – Port of Embarkation: C – Cherbourg, Q – Queenstown, S – Southampton.

And the target, **"Survival"**, where 1 represents that the passenger survived, and 0 - they did not.

The data cleaning followed similar steps described in Task 1 with the same reasoning: to maximise the effectiveness of data and reduce the potential of data leaks. The dataset was split into Train and Test in the ration of 750:140. The missing values were calculated based on the Train set and were applied to both datasets. The duplicates were removed.

However, the "Name" and "Ticket No." columns were dropped due to their lack of application – they do not provide relevant information to determine the survivability of a passenger. While one could argue that the sitting arrangement of passengers could affect their chances, extracting this information from "Ticket No." is more complicated and is redundant since there is a "Pclass" column already.

It is worth noting that almost 20% of the "Age" column was missing. While it has been filled with median, this incompleteness of the original dataset could lead to poorer performance of the models.

Next, the features were separated from the target, "Survival", and stored in X_ and Y_ datasets, respectively.

Thereupon, object datatype columns had to be encoded for SVM and KNN to be able to work with this data. One-hot encoding method was chosen due to the limited number of unique values in object columns, mostly ranging from two to four unique values. The original columns, "Sex" and "Embarked" were replaced by five new columns, where each column was named after a unique value from a column in the original dataset. They were filled with 0s and 1s where 1 represents that the data point belonged to that category, and 0 – it did not.

Finally, the features "Age", "Fare", "SibSP" and "Parch" were standardised. While it is essential for KNN and SVM, NN does not typically require standardisation. However, the operation could insignificantly improve the model too.

3.2. Methodology

KNN, a simple yet effective classifier, works by calculating the distance of a given data point to its k nearest neighbours, typically using the Euclidean distance formula:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

SVM, however, separates classes by identifying a hyperplane that best separates them. This hyperplane would have to maximise the distance between the closest data points from each class.

$$\min_{w,b} \frac{1}{2}\|w\|^2, \quad \text{subject to} \quad y_i(w^T x_i - b) \geq 1, \quad \forall i \in \{1, \ldots, n\}$$

Where:

- w – the weight vector.
- b – the bias.
- y_i – class labels.

And unlike the two other models, NN does not compute the distance between the points. Rather, it consists of layers of neurons with activation functions which models the complex decision boundaries. After being passed through multiple layers of neurons, the model provides a classification in the output layer. The weights are updated through back propagation and gradient descent. The formula for the forward propagation of a neuron is:

$$z = wx + b, \quad a = \sigma(z)$$

- z – the weighted sum of inputs.
- w – the weight.
- x – the input features.
- b – the bias.
- $\sigma(z)$ – an activation function.
- a – the output of a neuron.

NN was chosen as the main model due to its enhanced ability to capture non-linear relationships. While SVM can apply the kernel trick to fit the maximum-margin hyperplane in a transformed feature space by replacing the dot products with kernels, NN requires less tweaking, providing similar results. This hypothesis will also be tested in this case study. Consequently, SVM and KNN were chosen as the baseline models.

3.3. Experiments

The study conducts two rounds of experiments as in Task 1. In the first round, the models are built with no hyperparameter optimisation (i.e., models have their default parameters), and in the second stage, the models are optimised by Optuna, which uses the Bayesian optimiser described in the Regression task.

3.3.1. Experimental Settings

The default settings for all basic models can be found in their respective documentation. To fine-tune the parameters in a model, one has to provide the set of parameters to be tuned.

For KNN, these were:

- $n_{\text{neighbours}}$: 1, 20 – the range of the nearest neighbours.
- `weights`: `['uniform', 'distance']` – how much each neighbor contributes to the decision.
- `metric`: `['euclidean', 'manhattan', 'chebyshev']` – how distances are computed.
- `algorithm`: `['auto', 'ball_tree', 'kd_tree', 'brute']` – how neighbors are searched for efficiency.

For the SVM, these were:

- `trial.suggest_float`: `'C'`, 1e-5, 1e5, log=True – control the trade-off between margin maximisation and minimising classification error.
- `kernel`: `['linear', 'poly', 'rbf', 'sigmoid']` – how input data is mapped into a higher-dimensional space.
- `gamma`: `['scale', 'auto']` – how much influence each training example has.
- `degree`: (1, 5) if `kernel == 'poly'` else 3 – how complex the decision boundaries are.
- `class_weight`: `[None, 'balanced']` – adjust the weights for imbalanced datasets.

Finally, due to the different architecture of NNs, the model had different parameters to be fine-tuned:

- `num_layers`: 1, 3 – the number of hidden layers.
- `num_neurons_l{i+1}`: 32, 128, log=True – the number of neurons in each layer.

The activation function could be optimised in addition to the above parameters, but as Chaudhary M. suggests, `relu` is typically used for the hidden layers and `softmax/sigmoid` for the output layer.

To note, all optimisations used five-fold cross-validation except for SVM, which used the three-fold one. This was done to increase the usability of the limited data and to improve the performance of the models by exposing them to various

train-validation combinations.

3.3.2. RESULTS

To comprehensively assess the performance of the models, several evaluation metrics were chosen:

- **Accuracy** = $\frac{TP+TN}{\text{All predictions}}$ – assesses the proportion of correct predictions.
- **F-1 Score** = $\frac{2\cdot(\text{Precision}\cdot\text{Recall})}{\text{Precision}+\text{Recall}}$ – harmonic mean of Precision and Recall, useful when dealing with imbalanced datasets.
- **Precision** = $\frac{TP}{TP+FP}$ – the proportion of TP of all positive predictions, useful when the cost of FP is high (but here used to calculate F-1 score).
- **Recall** = $\frac{TP}{TP+FP}$ – how many positives are correct, useful when the cost of FN is high (i.e., missing a survivor, and thus, not saving them).
- **ROC AUC** = Receiver Operating Characteristics – Area Under the Curve. Useful to evaluate a model across all metrics.

The following tables show the results of the basic (default) models (Table 3) and the models after hyperparameter tuning (Table 4).

Model Metric	Accuracy	F-1 Score	Precision	Recall	ROC AUC
KNN	0.8071	0.7158	0.7556	0.68	0.7789
SVM	0.7929	0.7010	0.7234	0.68	0.7678
Neural Network	0.8214	0.7312	0.7907	0.68	0.7900

Table 3. Performance of the basic models on the "Titanic" dataset

Model Metric	Accuracy	F-1 Score	Precision	Recall	ROC AUC
KNN	0.8214	0.7423	0.7659	0.72	0.7989
SVM	0.7929	0.7010	0.7234	0.68	0.7678
Neural Network	0.8429	0.7556	0.8500	0.68	0.8067
NN (optuna)	0.8286	0.7391	0.8095	0.68	0.8464

Table 4. Performance of the fine-tuned models on the "Titanic" dataset

3.3.3. DISCUSSION

As can be observed from the tables above, all models seem to be a good fit for the given dataset, even with the default parameters. Parameter fine-tuning yielded small improvements. However, considering that this is a situation of life and death, improvements of even a fraction of a percentage are crucial.

As predicted in the hypothesis above, NN was better at capturing non-linear relationships between the features, albeit with a smaller lead. Nonetheless, fine-tuned KNN had a better Recall rate than NN. As described above, the cost of missing a survivor could be tragically high. It is the main limiting factor of NN in this case study. This could be due to the 20% of "Age" column being missing or the model's architecture. More tests would have to be performed on different dataset to prove/refute this hypothesis.

4. Conclusion

In conclusion, LR, RF and XGB are powerful ML models to evaluate the price of a house based on various factors, with XGB having a slight edge over the other models with smaller errors. In contrast, KNN, SVM and NN proved useful when trying to decide whether one would survive the Titanic crash, with NN having better accuracy, while KNN had higher Recall. Overall, the case studies demonstrated some applications of AI in regression and classification tasks. The main limitation was the complexity of the models and their required computational power for training. More complex models could be explored in future studies.